# TYING UP LOOSE THREADS: MAKING YOUR PROJECT NO-GIL READY

Charlie Lin

# Introduction

# Simple module rand.c

```
static PyObject* pyrand(PyObject*
self, PyObject* arg) {
  srand(time(NULL));
  return PyLong_FromLong(rand());
}

static PyMethodDef methods[] = {{
    "myrandom", (PyCFunction)
pyrand, METH_NOARGS,
    “Return a random integer.”},
   {NULL, NULL, 0, NULL}
};
```

```
PyModuleDef rand module = {
    PyModuleDef_HEAD_INIT, "myrand",
NULL, ,-1,
    methods, NULL, NULL, NULL, NULL
};

PyMODINIT_FUNC PyInit_rand(void)
{
    return
PyModule_Create(&rand_module);
}
```

# Simple module setup.py

```
from setuptools import setup, find_packages, Extension

setup(
    name='fib', version='0.1.0', packages=find_packages(),
icense='GPL-2',
    ext modules=[
        Extension{
            'myrand',
            ['rand.c'],
        },
    ],
    classifiers=[ Development Status :: 3 - Alpha'],
)
```

# Simple module

```
python setup.py build_ext --inplace
```

Result:

```
>>> import myrand
>>> myrand.myrandom()
-5
>>>
```

# What is the GIL?

Only 1 thread can run Python code

- Concurrent access to objects

Extensions can manually release GIL

# Reasons to Rid the GIL

## Existing appraches:

- Multiprocessing: overhead of starting interpreter per subprocess, serialization to shared memory required
- Manual release through C-API: not scalable

→

Difficult to scale for nunber-crunching, cumbersome to workaround (bottleneck with single-digit threads)

# Free-threading

# What is the free-threaded interpreter?

| | |
|---|---|
| Many simultaneous threads! PEP 703 | minmalloc: GC objects without linked list & lighter locks for builtin types |
| | Refcount changes |
| | Container thread-safety |
| | C API enhancements |

# Porting tips

# C/C++/Rust code…

## Multi-phase

```
static PyModuleDef_Slot module_slots[]
= {
...
#ifdef Py_GIL_DISABLED
{Py_mod_gil, Py_MOD_GIL_NOT_USED},
#endif
{0, NULL},
};
static struct PyModuleDef moduledef = {
   PyModuleDef_HEAD_INIT,
   .m_slots = module_slots,
   ...
};
```

## Single-phase

```
PyMODINIT_FUNC PyInit__module(void) {
PyObject *mod = PyModule_Create(&module);
if (mod == NULL) { return NULL; }
#ifdef Py_GIL_DISABLED
PyUnstable_Module_SetGIL(mod,
Py_MOD_GIL_NOT_USED);
#endif
return mod;
}
```

# C/C++/Rust code…

## setuptools

```
from Cython.Compiler.Version import
version as cython_version

from packaging.version import Version

compiler_directives = {}

if Version(cython_version) >=
Version("3.1.0"):
compiler_directives["freethreading_comp
atible"] = True

setup( ext_modules=cythonize(
extensions,
compiler_directives=compiler_directives
, ) )
```

## meson

```
cy =
meson.get_compiler('cython')

if
cy.version().version_compare('>=
3.1.0')
add_project_arguments('-Xfreethr
eading_compatible=true',
language : 'cython')

endif
```

# C/C++/Rust code…

pybind11

```
#include <pybind11/pybind11.h>

namespace py = pybind11;

PYBIND11_MODULE(example, m,
py::mod_gil_not_used()) {

   ...

}
```

PyO3

```
#[pymodule(gil_used = false)]

fn my_module(py: Python, m:
&Bound<'_, PyModule>) ->
PyResult<()> {

   ...

}
```

# C/C++/Rust code…

## Global to thread-local state

- Python and Cython: threading.local.
- C/C++: Py_tss API: thread-local Python objects or thread_local for native objects

## Global cache

- Disable if not performance-critical
- Cache in PyInit (single-phase only)

```
#ifdef Py_GIL_DISABLED
static PyMutex cache_lock = {0};
#define LOCK() PyMutex_Lock(&cache_lock)
#define UNLOCK()
PyMutex_Unlock(&cache_lock)
#else
#define LOCK()
#define UNLOCK()
#endif
static int *cache = NULL;
static PyObject *global_table = NULL;
int initialize_table(void) { // called
during module initialization
global_table = PyDict_New(); return; }
int function_accessing_the_cache(void) {
LOCK(); // use the cache UNLOCK(); }
```

# C API

- PyListObject, PyDictObject, and PySetObject: internal lock
  - PyDict_Next(): exception

```
Py_BEGIN_CRITICAL_SECTION(dict);
PyObject *key, *value;
Py_ssize_t pos = 0;
while (PyDict_Next(dict, &pos, &key,
    &value)) {
  ...
}
Py_END_CRITICAL_SECTION();
```

- *Critical section: Python thread acquires lock to acquire (implicit) PyMutex*

```
// read the count, returns new
    reference to internal count value
PyObject *result;
Py_BEGIN_CRITICAL_SECTION(obj);
result = Py_NewRef(obj->count);
Py_END_CRITICAL_SECTION();
return result;


// write the count, consumes reference
    from new_count
Py_BEGIN_CRITICAL_SECTION(obj);
obj->count = new_count;
Py_END_CRITICAL_SECTION();
```

# CI

## Setup/actions

```
jobs:
  free-threaded:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@...
      - uses: actions/setup-python@...
        with:
          python-version: 3.14t
          freethreading: true
```

## cibuildwheel

```
- name: Build wheels
    uses: pypa/cibuildwheel@...
    env:
      CIBW_ENABLE:
 cpython-freethreading
      CIBW_BUILD: cp314t-${{
 matrix.buildplat }}
```

# Tests

Unit testing: pytest-run-parallel, unittest-ft

Functionality that is known not to be thread-safe includes:

- pytest.warns on Python 3.13 and older, it relies on warnings.catch_warnings, which is not thread-safe until Python 3.14, and even then only on the free-threaded build by default.
- The tmp_path and tmpdir fixtures, since they rely on the filesystem
- The capsys fixture, because of shared use of sys.stdout and sys.stderr.
- The monkeypatch fixture, since monkeypatching inherently modifies global state in a class or module.
- TSan, ASan

# Tests

```
# Setting PYTHON_GIL=0 ensures that the GIL is effectively disabled.
PYTHON_GIL=0 gdb --args python my_program.py --args ...
# To test under pytest PYTHON_GIL=0 gdb --args python -m pytest -x -v
"test_here.py::TestClass::test_method“
# Using LLDB (under LLVM/clang) PYTHON_GIL=0 lldb -- $(which python)
my_program.py
# Using LLDB (and pyenv) PYTHON_GIL=0 lldb -- $(pyenv which python)
$(pyenv which pytest) -x -v "test_here.py::TestClass::test_method"
```

- PYTHON_GIL=0 cygdb build/cp314td/ -- --args python -m pytest -x -v skimage/

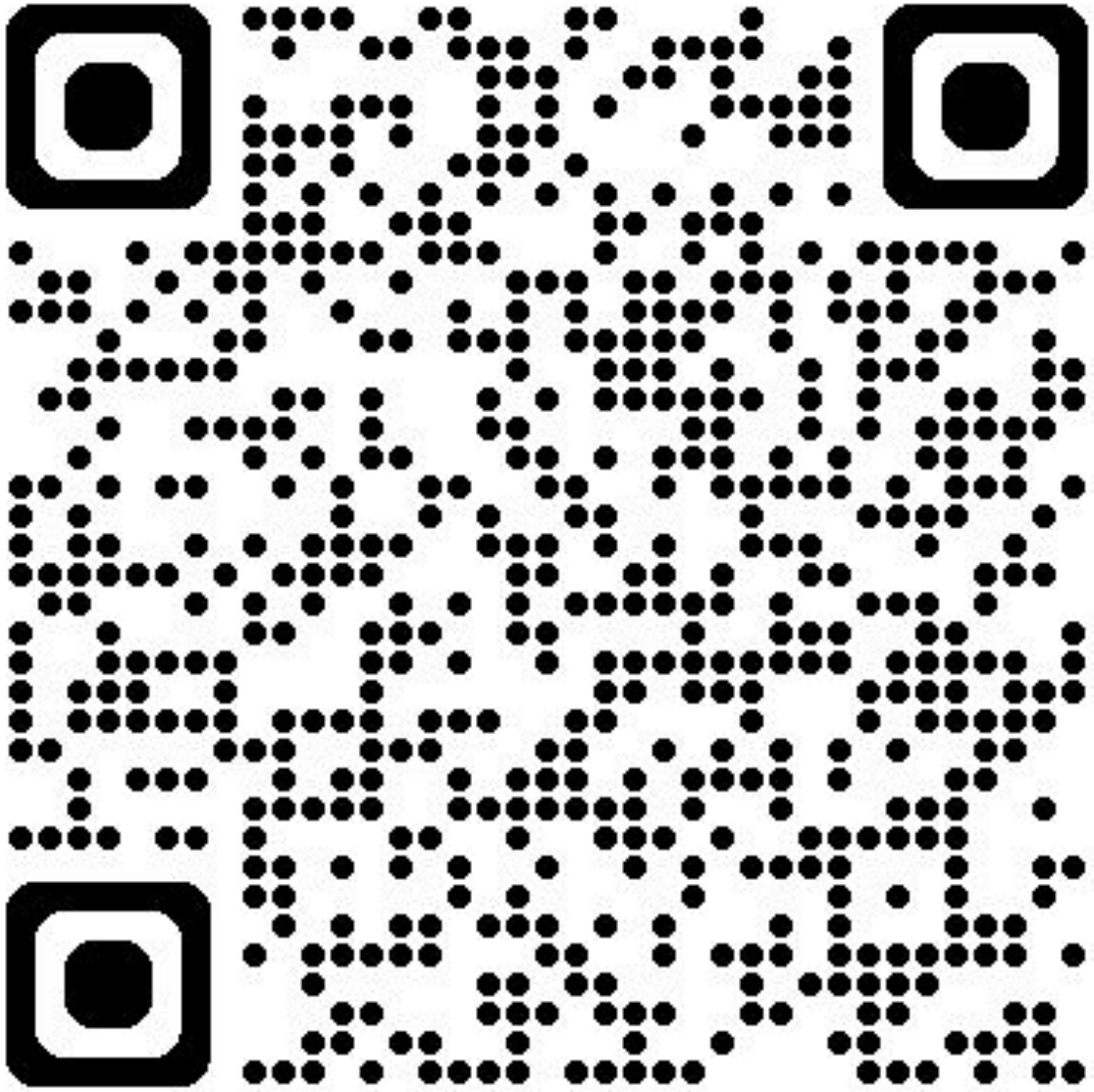
- Cpython_sanity: Docker imgs of Python interpreters w TSan and ASan

# Other helpful stuff

github.com/iskandergaba/free-threading.git: wraps threading (no-GIL) and multiprocessing (GIL)

github.com/python/pythoncapi-compat: official polyfill

# Closing remarks

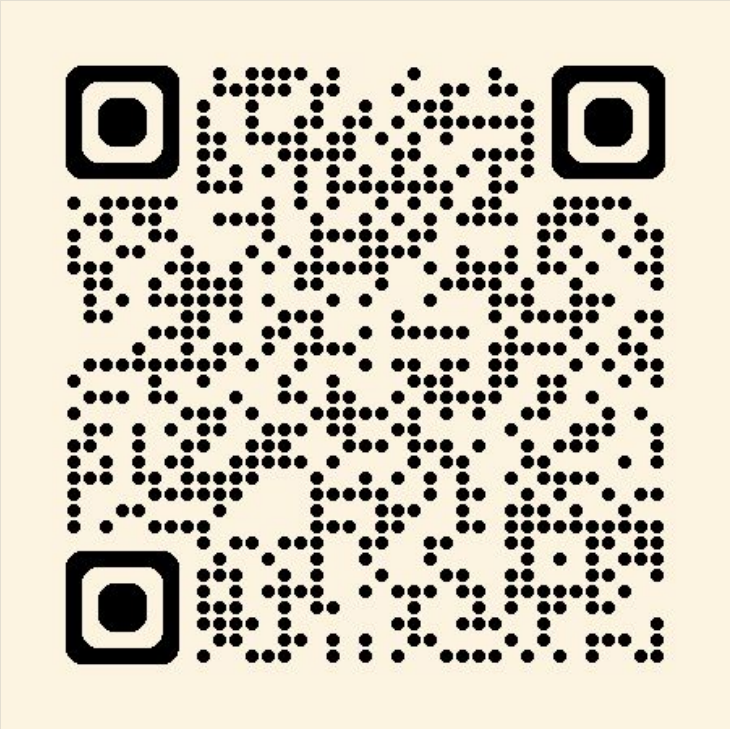


## Current progress

55% of most popular wheels in PyPI is free-threaded compatible!

# Closing remarks

Proof:



Bump dependencies!

- https://py-free-threading.github.io/
- https://docs.python.org/3/howto/free-threading-extensions.html#freethreading-extensions-howto
- https://peps.python.org/pep-0703/#overview-of-cpython-changes
- https://docs.python.org/3/howto/free-threading-python.html#freethreading-python-howto
- C API Extension Support for Free Threading — Python 3.14.4 documentation


# Q & A

PLEASE ASK AWAY!